

# Final report: Stanford RNA 3D Folding

|                      |                        |                         |
|----------------------|------------------------|-------------------------|
| <b>Max Neuwinger</b> | <b>Sarah Verreault</b> | <b>Andreas Hiropedi</b> |
| ETH Zürich           | ETH Zürich             | ETH Zürich              |
| mneuwinger@ethz.ch   | sverreault@ethz.ch     | ahiropedi@ethz.ch       |

## Abstract

The computational prediction of an RNA molecule's 3D structure from its 1D sequence remains a central challenge in structural biology. This report details our work within the Stanford 3D RNA Folding Kaggle competition, where we investigate two distinct strategies to improve upon state-of-the-art predictors. First, we specialized the generalist biomolecule model Protenix by engineering a pipeline to integrate Multiple Sequence Alignment (MSA) data and fine-tuning the model on a recent RNA dataset. Second, motivated by the observation that no single model excels on all targets, we developed a meta-learning ensemble that uses 28 engineered features to predict the quality of candidates from a diverse pool of base models, enabling an intelligent selection of the final structure. Both strategies yielded significant performance gains over their respective baselines, validating targeted fine-tuning and learned ensembling as powerful approaches for advancing automated RNA structure prediction.

## 1 Introduction

The three-dimensional structure of Ribonucleic acid (RNA) is fundamental to its diverse biological functions, from regulating gene expression to catalyzing biochemical reactions (Leamy et al., 2016). However, experimentally determining these structures is a costly and low-throughput process. Consequently, the computational prediction of RNA 3D structure from its 1D nucleotide sequence remains one of biology's grand challenges (Wang et al., 2025). Recent years have seen breakthroughs in protein structure prediction, exemplified by AlphaFold (Abramson et al., 2024), which have inspired a new generation of deep learning models for RNA. Yet, progress in the RNA domain has been slower, hindered by limited structural data and the inherent complexity and flexibility of RNA molecules (Wang et al., 2025).

Current state-of-the-art models often struggle to generalize to novel RNA families or artificially designed sequences, and their performance is frequently surpassed by human experts who leverage deep domain knowledge and analyze co-evolutionary data from Multiple Sequence Alignments (MSAs) (He et al., 2025). This performance gap highlights a critical opportunity for research. The Stanford 3D RNA Prediction Kaggle competition (He et al., 2025) provides a unique platform to address this gap, framing the problem as a community-wide scientific endeavor. The competition organizers aim to synthesize the top-performing models into a new, state-of-the-art method, echoing a previous successful community effort for 2D RNA prediction.

Our project situates itself within this collaborative context. We do not aim to develop a revolutionary, standalone model from scratch, as this would require computational resources far beyond our means. Given the persistent challenges in RNA 3D structure prediction, particularly concerning generalizability and the integration of evolutionary data, our research addresses the question: **Can we incrementally improve upon existing state-of-the-art models for RNA 3D prediction through targeted, resource-aware fine-tuning and ensembling strategies?** We hypothesize that even without access to massive computational clusters, carefully designed experiments can yield valuable insights and contribute to the community's collective goal.

To investigate this, we pursue two distinct but complementary research avenues:

**(1) Specializing a Generalist Model:** We hypothesize that a general-purpose biomolecule model like Protenix (an open-source AlphaFold3 implementation) can be improved for the RNA-specific task. Our approach involves fine-tuning the model on the latest competition data and, critically, engineer-

ing a pipeline to integrate MSA information—a rich data source the base model did not leverage for RNA. This tests whether targeted specialization can unlock greater predictive power from a large, pre-trained foundation model.

**(2) Ensemble Modeling:** We explore strategies to combine predictions from diverse models. We hypothesize that by leveraging the complementary strengths of different architectures we can produce more robust and accurate final predictions than any single model can achieve alone.

## 2 Related Work

### 2.1 State-of-the-Art in RNA 3D Prediction

Recent advancements in 3D RNA structure prediction are primarily driven by deep learning approaches, but comprehensive benchmarks highlight persistent challenges such as accurately modeling long sequences, non-canonical interactions, local structural features, as well as managing the complex role of Multiple Sequence Alignments (MSAs) (Bernard et al., 2024; Nithin et al., 2024; Bahai et al., 2024) to name a few. Overcoming these difficulties has led to the development of several state-of-the-art models using a wide range of architectural approaches.

The generalist model **AlphaFold 3** (Abramson et al., 2024) employs a multi-modal diffusion architecture for biomolecule prediction, achieving strong results on benchmarks like CASP15, yet still struggling with certain RNA-specific complexities. In contrast, models like **DRfold2** (Li et al., 2025) and **RhoFold+** (Shen et al., 2024) integrate specialized RNA language models with end-to-end geometric learning to achieve high accuracy, performing particularly well on benchmarks like CASP16 and RNA-Puzzles. A third paradigm, employed by **trRosettaRNA** (Wang et al., 2023) and **DeepFoldRNA** (Thill, 2025), first uses transformers to predict geometric restraints and then leverages these as constraints for optimization-based folding, thus balancing computational efficiency and accuracy across RNA-Puzzles and CASP evaluations.

Finally, models like **RibonanzaNet2** (He et al., 2024) represent an alternative approach, trained as foundation models on chemical reactivity data (SHAPE). This provides a different inductive bias compared to purely structure-based models. It is worth noting, however, that RibonanzaNet2 is not explicitly state-of-the-art and is an alpha release that builds upon the original RibonanzaNet archi-

tecture (He et al., 2024) and is currently under evaluation via the Kaggle competition.

The diversity of these successful but distinct approaches validates our core hypothesis that no single model has definitively solved the 3D RNA folding problem, motivating the exploration of targeted fine-tuning and ensemble strategies.

### 2.2 Strategies for Model Improvement

Beyond developing new architectures, two primary strategies exist for improving predictive accuracy: specializing a single model via **fine-tuning**, and combining the outputs of multiple models via **ensembling**.

Fine-tuning has proven effective for adapting large pre-trained models to specific biological domains. While parameter-efficient fine-tuning (PEFT) techniques like LoRA (Low Rank Adaptation) have shown promise for adapting protein language models with minimal compute overhead (Hu et al., 2022; Schmirler et al., 2024), our project focuses on the potential of direct, full fine-tuning of a generalist model.

The use of ensembles to improve upon the predictions of single models is also well-established. This strategy typically involves two steps: generating a pool of candidate structures and then using a scoring function to rank them. For example, the FebRNA method generates vast ensembles that are then ranked using a specialized, high-performance scoring function to select the final model (Zhou et al., 2022). The critical challenge of ranking has also motivated the development of **learned quality assessment** metrics. The trRosettaRNA pipeline, for instance, trains a regression model to produce an "estimated RMSD" (eRMSD) score, demonstrating the principle of training a model to predict the quality of its own outputs (Wang et al., 2023). Finally, the core rationale for combining different predictors is supported by comparative studies showing that state-of-the-art models possess distinct, complementary strengths, making a single best model elusive (Nithin et al., 2024). This body of work directly validates exploring both full fine-tuning and a learned quality assessment model to select from a pool of diverse candidates.

## 3 Data

The primary datasets for this project were provided by the Stanford 3D RNA Folding Kaggle competition (He et al., 2025). The competition is structured

in three phases, culminating in a final evaluation on a hidden test set of future RNA targets to ensure model generalization.

By the time of our experiments, the competition hosts had released two versions of the training data: *V1 Dataset (train\_sequences.csv)*: The original training set consisting of 844 RNA sequences, with a median length of 39.5 nucleotides.

*V2 Dataset (train\_sequences.v2.csv)*: An expanded set of 5,135 sequences extracted directly from the Protein Data Bank (PDB) using a “relaxed filter” for unstructured RNAs. This provided a much larger and more diverse dataset with a median length of 98 nucleotides.

Given that the V2 dataset represented a comprehensive and recent extraction from the authoritative PDB, we chose to focus our efforts exclusively on this official data source rather than introducing external datasets. The competition also provided Multiple Sequence Alignments (MSAs) in FASTA format for each training target.

### 3.1 Adapted Evaluation Strategy

Due to unforeseen logistical challenges with the Kaggle competition platform, which prevented evaluation on the official hidden test set within our project’s timeframe, we developed a custom test set to ensure rigorous and timely model assessment. The full details of these evaluation challenges are documented in Appendix A.

The official validation set provided by the competition was unsuitable for our purposes, as its small size (12 structures) and age (from the 2022 CASP15 challenge) made it unrepresentative for models trained on the much larger and newer V2 dataset. Our revised strategy involved a temporal split of the `train_sequences.v2.csv` dataset. Structures with a release date on or after February 1, 2025, were designated for our new test set, resulting in a custom set of 94 diverse and recent RNA structures.

To prevent data leakage, we verified that our chosen test samples were chronologically newer than the training data cut-off dates of the baseline models we evaluated: Protenix (September 30, 2021), DRfold2 (April 1, 2022), and RibonanzaNet2 (early 2024). Further analysis of the adapted dataset can be found in Appendix E.

## 4 Methodology

In this section, we describe our methodological approach for the two main directions explored in the project: (1) fine-tuning and adapting an existing generalist model (Protenix) for RNA-specific tasks, and (2) building ensemble strategies that combine predictions from multiple models to improve robustness and accuracy.

### 4.1 Specialized Fine-Tuning of Protenix with MSA Integration

Our approach involved specializing *Protenix*, an open-source implementation of the generalist AlphaFold3 architecture (Abramson et al., 2024), for the specific task of RNA 3D structure prediction. A key design principle of AlphaFold3 was to reduce the model’s reliance on deep multiple-sequence alignments (MSAs), which were central to its predecessor’s success. In line with this, the base *Protenix* model does not leverage MSA information for RNA predictions, treating it as a generalist tool. Recent studies suggest, however, that high-quality, RNA-specific MSAs can still provide complementary long-range covariation signals that translate into measurable improvements in tertiary-structure accuracy—particularly when homologous sequence depth is adequate and alignment noise is controlled (Zhang et al., 2024; Szikszai et al., 2025). Guided by these insights, our core hypothesis was that we could enhance *Protenix*’s predictive accuracy for RNA by (1) implementing a bespoke data pipeline to re-integrate evolutionary information from MSAs and (2) fine-tuning the model on this enriched, chronologically newer dataset.

### 4.2 Data Pipeline and MSA Integration

A significant engineering effort was required to bridge the gap between the competition’s data format and the specific input requirements of Protenix. This involved building a custom data pipeline, preprocessing the provided MSAs, and adapting the model’s internal featurization code.

First, we partitioned the `train_sequences.v2.csv` dataset to create dedicated training and validation sets for the fine-tuning procedure. Using a temporal split with a cut-off date of October 1, 2024, we generated a training set of approximately 4,300 structures and a validation set of 276 structures, which was used exclusively to monitor training progress. In parallel, we preprocessed the competition’s



FASTA-formatted MSAs by converting them to the required Stockholm (.sto) format and creating a `seq_to_target_map.json` index, as required by the Protenix model for data ingestion.

These components were orchestrated by a custom PyTorch Dataset class, `KaggleRNADataset`. This class loads the data, performs integrity checks, and feeds samples to the model. A key function of this class was to utilize Protenix’s native dynamic cropping module.

Finally, to enable the model to process this new data stream, we directly modified its source code. The model’s original `MSAFeaturizer` was designed only for proteins. We extended its functionality by implementing a new `RNAMSAFeaturizer` class capable of parsing our preprocessed Stockholm files.

#### 4.2.1 Fine-Tuning Protocol

Our fine-tuning experiments were conducted using four NVIDIA RTX 4090 GPUs. The protocol was designed to closely emulate the Fine tuning stage 2, as detailed in the official Protenix documentation.

We initialized the model with the official pre-trained weights from Protenix v0.2.0. To manage memory, we used a reduced `train_crop_size` of 256 (compared to 768) and utilized `bfloat16` mixed-precision for training. The effective batch size was 32, achieved with a per-GPU `diffusion_batch_size` of 8 and gradient accumulation over 2 steps. We employed the AdamW optimizer with a learning rate of  $1 \times 10^{-4}$  and a linear warmup over the first 1000 steps.

Following the “Fine tuning 2” protocol, the composite loss function was configured to prioritize generative and geometric objectives. Key weights included a high value for the diffusion loss ( $\alpha_{\text{diffusion}} = 4.0$ ), complemented by the distogram ( $\alpha_{\text{distogram}} = 0.03$ ), bond ( $\alpha_{\text{bond}} = 1.0$ ), and confidence ( $\alpha_{\text{confidence}} = 1 \times 10^{-4}$ ) losses. The pairwise aligned error (PAE) and smooth L1 IDDT loss terms were disabled for this stage ( $\alpha_{\text{pae}} = 0.0$ ,  $\alpha_{\text{smooth\_iddt}} = 0.0$ ).

The model was trained for a total of 48 hours. During this process, we saved checkpoints every 1000 steps to monitor the validation loss.

### 4.3 Model Ensembles

We explored both simple heuristic-based and learned ensemble strategies to improve predictive accuracy. Our preliminary analysis revealed that

no single model consistently outperformed others across all target structures. Instead, the highest-scoring prediction for a given target often originated from different base models (Protenix, DR-fold2, RibonanzaNet2), suggesting their strengths are complementary. This observation strongly motivated our development of ensemble methods to intelligently combine their outputs.

To support these strategies, we leveraged the model-provided predicted Local Distance Difference Test (pLDDT) scores as a per-residue confidence measure  $c_i^{(m)}$  for each model  $m$ , as defined by Mariani et al. (2013).

#### 4.3.1 Heuristic Ensembles

Our initial approaches involved several heuristic-based strategies for combining model outputs. Formulas and detailed explanations for each method are provided in Appendix D. These strategies include:

**Naive Ensemble:** Selects the first three predictions from Protenix and the first two from DRfold2. **Simple Average Ensemble:** Computes the average of all predictions across all models without weighting. **Weighted Average Ensemble:** Computes a confidence-weighted average of model predictions. **Best Confidence Ensemble:** Selects the prediction with the highest confidence for each nucleotide. **Squared Weights Ensemble:** Modulates weights using the squared confidence score. **Dynamic Ensemble:** Uses the distribution of confidence scores to alternate between best confidence, weighted average, and defaulting to Protenix.

These heuristics lack flexibility and adaptivity, and are sensitive to noise. The heuristics are particularly vulnerable to over confidence and poor calibration which occurs with data contamination.

#### 4.3.2 Meta-Learning Ensemble

Given the limited success of simple heuristic ensembles, we developed a more sophisticated meta-learning approach. The goal was to train a model to predict the final TM-score for any given candidate structure, allowing us to rank all predictions and select the most promising ones.

**Feature Engineering.** The model’s performance relies on a set of 28 engineered features, designed to capture diverse structural properties. The development of this feature set was iterative and guided by an analysis of model biases. We discovered that the meta-learner was prone to overestimating the

quality of RibonanzaNet2 candidates due to data contamination in its training set. To counteract this, our final feature set was designed around three principles: (1) model-agnostic physical and geometric properties, such as a custom energy score; (2) ensemble consensus metrics, like the average RMSD to other candidates; and (3) targeted interaction features, which allow the meta-learner to calibrate its interpretation of a metric (e.g., confidence) based on its source model. A full list of these features is provided in Appendix C.

**Architecture and Training.** Our final architecture is a regularized, fully-connected neural network. It takes the 28 standardized features as input and passes them through three hidden layers (64, 32, and 16 units) with ReLU activations. To prevent overfitting, dropout layers ( $p=0.4$ ) follow the first two hidden layers. The output layer is a single neuron with a sigmoid activation to produce a predicted TM-score in the range  $[0, 1]$ .

The model was trained to minimize Mean Squared Error (MSE) loss using the Adam optimizer with a weight decay of  $1 \times 10^{-4}$ . We employed an early stopping mechanism, halting training if the validation loss failed to improve for 10 consecutive epochs. The training data consists of individual candidate structures from all three base models, with their ground-truth TM-scores (calculated via US-align) serving as the regression target.

## 5 Results and Discussion

This section presents the empirical results of our two primary experimental avenues.

### 5.1 Baseline Model Performance

To ground our work, we first established baseline performance for several state-of-the-art models by submitting their predictions to the official Kaggle public leaderboard prior to its closure. The results of these baseline evaluations, using the official best-of-5 TM-score metric, are summarized in Table 1.

While the absolute scores on the public leaderboard are slightly lower than those observed on our custom test set (due to differences in the target structures), they are within the same general performance range and, critically, follow the same relative ranking. Protenix emerges as the strongest baseline, validating its selection as the primary candidate for our fine-tuning efforts. Conversely, RhoFold’s significantly lower score justified its exclusion from further experiments. This consistency

Table 1: Baseline model performance on the official Kaggle public leaderboard and our custom test set.

| Model         | Public LB<br>TM-score | Custom<br>TM-score |
|---------------|-----------------------|--------------------|
| RhoFold       | 0.102                 | -                  |
| RibonanzaNet2 | 0.295                 | 0.319              |
| DRfold2       | 0.315                 | 0.354              |
| Protenix      | <b>0.354</b>          | <b>0.404</b>       |

between the official leaderboard and our custom test set provides confidence that our internal evaluation framework is a valid and representative proxy for assessing model performance on this task.

### 5.2 ProteinX Fine-Tuning and MSA Integration

Our first major experimental approach involved specializing the ProteinX model. To disentangle the effects of our strategy, we performed two parallel fine-tuning runs: one with our full MSA-enriched data pipeline, and an ablation run using the exact same protocol but with the MSA features disabled. As the MSA data processing adds significant computational overhead, the non-MSA run was able to complete more training steps in the same 48-hour period.

The training dynamics for both runs proceeded as anticipated. As illustrated for the MSA-enabled run in Figure 1, the training loss exhibits a stable, monotonic decrease, while the validation loss improves overall despite some volatility, confirming that the models were successfully learning from the new data.

The quantitative impact of these experiments is summarized in Table 2, which compares the performance of the baseline model against both fine-tuned versions at different training checkpoints, evaluated on our custom 94-sample test set.

The results yield several key insights. First and most critically, **all fine-tuned variants dramatically outperform the pretrained baseline**, confirming that specializing a generalist model on domain-specific data is a highly effective strategy.

Second, the results reveal a nuanced trade-off between incorporating MSA data and training for longer. At a comparable training checkpoint ( $\sim 6k$  steps), the EMA model with MSA (0.507) clearly outperforms its non-MSA counterpart (0.491), suggesting that MSA data provides a valuable induc-



Figure 1: Training and validation loss curves for the ProteinX fine-tuning run with MSAs.

tive bias that accelerates learning. However, by training the computationally cheaper non-MSA model for longer ( $\sim 10k$  steps), we achieved our best overall result of **0.509**.

The "Mean Prediction Score," which averages the TM-score across all five predictions per target, provides further insight into model reliability. Our two top-performing models—the long-trained non-MSA model and the MSA-enabled model—achieved nearly identical mean scores (0.474 and 0.471, respectively). This indicates that both are not just producing a single lucky high-quality prediction but are consistently generating a robust set of plausible structures.

This total gain of nearly 0.08 points over the original baseline is highly significant; on the competitive official Kaggle leaderboard, an improvement of this magnitude would represent a leap from a mid-tier position to a top-tier rank. This presents a critical trade-off between computational efficiency and data richness. Within our fixed time budget, the greater number of training updates for the non-MSA model led to the highest score. However, this faster convergence may come at the cost of long-term potential; a model trained without the constraints of evolutionary data is more susceptible to overfitting on the training set, which could

Table 2: Performance of ProteinX variants on our 94-sample test set. This ablation study compares fine-tuning with and without MSAs at different training durations.

| Model Configuration                                        | Test TM-score | Mean Pred. Score |
|------------------------------------------------------------|---------------|------------------|
| Pretrained (no MSA)                                        | 0.430         | -                |
| <i>Fine-tuned, checkpoint <math>\sim 6k</math> steps:</i>  |               |                  |
| No MSA (Raw)                                               | 0.495         | 0.457            |
| No MSA (EMA)                                               | 0.491         | 0.454            |
| With MSA (Raw)                                             | 0.486         | 0.450            |
| With MSA (EMA)                                             | 0.507         | 0.471            |
| <i>Fine-tuned, checkpoint <math>\sim 10k</math> steps:</i> |               |                  |
| No MSA (Raw)                                               | 0.506         | 0.471            |
| No MSA (EMA)                                               | <b>0.509</b>  | <b>0.474</b>     |

limit its peak performance. In contrast, the MSA-enabled model, while slower to train, is learning from a richer signal that acts as a powerful regularizer. This could prevent premature overfitting and potentially allow it to achieve a higher absolute performance ceiling if training were extended further.

### 5.3 Model Ensembles

We evaluated the various ensemble strategies described in the Methodology section by combining predictions from the Protenix model, DRfold2, and RibonanzaNet2. All evaluations were performed on our custom test set.

#### 5.3.1 Heuristic Ensembles

The performance of the simple, heuristic-based ensembles is summarized in Table 3. The results show that none of these strategies were able to outperform our best single model (Protenix, TM-score 0.430). The naive and dynamic ensemble performed best among the heuristics, but their scores of approximately 0.37 and 0.36 are lower than the Protenix baseline. This indicates that simple averaging or confidence-based heuristics are insufficient to effectively combine the outputs of these diverse models, which motivated our move to a learned approach. For the three models, we noted that **Protenix** was the most confident model, with a mean confidence score of 58.81%, followed by **DRfold2** with 51.50% and **RibonanzaNet2** with 50.80%.

| Model Ensemble   | TM-score      |
|------------------|---------------|
| Naive            | <b>0.3719</b> |
| Simple average   | 0.1809        |
| Weighted average | 0.1718        |
| Best confidence  | 0.3068        |
| Squared weights  | 0.3145        |
| Dynamic          | <b>0.3626</b> |

Table 3: Model ensemble performance. Scores are reported as the mean best-of-5 TM-score.

## 5.4 Meta Model

Given the poor performance of heuristic-based ensembles, we evaluated our meta-learning model, which was trained to directly predict the TM-score of each candidate structure. The model’s training converged after 86 epochs, with the best checkpoint saved at epoch 76 based on the validation loss, as shown in Figure 2. This section presents the performance of this learned ensemble, using the baseline versions of Protenix (TM-score: 0.430), RibonanzaNet2 (TM-score: 0.418), and DRfold2 (TM-score: 0.350) as inputs.

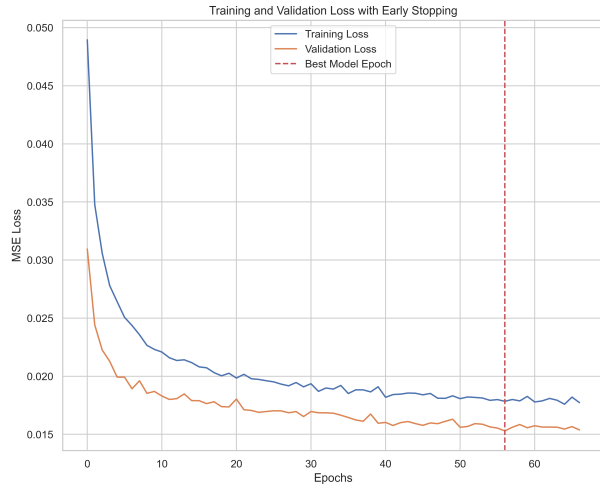


Figure 2: Training and validation loss curves for the meta-learning model. The red dashed line indicates the epoch (76) where the best-performing model was saved, just before the validation loss began to plateau.

### 5.4.1 Performance and Analysis

The final meta-model achieved a mean best-of-5 TM-score of **0.473** on our test set. This represents a significant improvement over all individual baseline models and is a substantial increase of over

0.10 points compared to the best-performing heuristic ensemble (0.371).

A detailed breakdown of the model’s performance reveals key insights (Table 4). The model achieves 94.1% of the “Oracle Score” of 0.502, which is the theoretical maximum performance achievable by perfectly selecting the single best structure from the candidate pool for each target. This indicates that the set of five structures selected by our model is, on average, of very high quality.

Table 4: Overall performance metrics for the final meta-learning model.

| Performance Metric              | Value        |
|---------------------------------|--------------|
| Oracle Score (Theoretical Max)  | 0.502        |
| <b>Achieved TM-Score</b>        | <b>0.473</b> |
| Performance vs. Oracle          | 94.1%        |
| Recall@5 (Best Candidate Found) | 54.3%        |

To validate our feature engineering process, we analyzed the feature importances from an identically configured *LightGBM* model, which performed comparably to our neural network but provided clearer interpretability. This analysis revealed the three most dominant signals for predicting structure quality:

**avg\_rmsd\_to\_others:** By a large margin, the most important feature was the average RMSD of a candidate to all others. This confirms that a structure’s agreement with the ensemble consensus is the strongest indicator of its validity.

**rog\_x\_Ribonanza:** The second most critical feature was the radius of gyration specifically for RibonanzaNet2 predictions. This shows the model learned a targeted rule: the compactness of a Ribonanza structure is a uniquely powerful predictor of its quality, likely to penalize the “exploded,” unrealistic structures sometimes produced by this model.

**solvation:** A model-agnostic measure of physical plausibility. This demonstrates the model successfully learned to reward structures that are well-packed and have a favorable solvation energy, a fundamental principle of protein and RNA folding.

### 5.4.2 Model Behavior and Calibration

Despite the strong overall performance, analysis of the model’s predictions reveals a persistent calibration issue stemming from the data contamination discussed in our methods. Table 5 shows that while the model is well-calibrated for



Protenix, it remains significantly overconfident in RibonanzaNet2, predicting an average score of 0.426 against a true average of only 0.319 on the test set.

Table 5: Meta-model calibration analysis on the test set.

| Base Model    | Predicted TM | True TM |
|---------------|--------------|---------|
| Protenix      | 0.397        | 0.404   |
| DRfold2       | 0.387        | 0.354   |
| RibonanzaNet2 | 0.426        | 0.319   |

This overconfidence directly influences the final selections. Despite its calibration issues, RibonanzaNet2 structures constitute a majority (53.8%) of the final top-5 ensemble, with Protenix and DRfold2 making up the remainder at 26.0% and 20.2%, respectively. Our experiments with mitigation techniques, such as weighted loss and post-processing penalties, counter-intuitively degraded performance. This surprising result leads to a crucial insight: while RibonanzaNet2 is often overestimated, it is not uniformly poor. It does provide the best structure for a subset of targets, and aggressively penalizing it prevents the model from making these correct selections. Our final model, therefore, represents a balance, leveraging general physical and ensemble features to achieve high performance despite the challenge of a partially unreliable input.

## 6 Limitations

One limitation of our study originated from the general scarcity of high-quality, experimentally-determined RNA 3D structures, a limitation present in all current research in this field. This data bottleneck limits the potential for any model to generalize perfectly, meaning that performance gains will likely continue as more data becomes available.

Additionally, given the project was shaped by the competition’s framework, this presented several limiting factors. The competition’s ”best-of-5” scoring metric rewards a model for producing at least one good prediction out of five, which does not necessarily reflect the real-world need for consistent and reliable accuracy. While we had access to significant computational resources, the sheer scale of modern models still placed practical limits on the extent of our hyperparameter search and the number of training runs we could perform. The unforeseen closure of the official submission portal

necessitated our pivot to a custom test set. While we constructed this set with temporal splitting to prevent data leakage, any custom set carries a risk of sampling bias and may not perfectly represent the diversity of structures in the hidden test set. To avoid out-of-memory errors, we applied a length cutoff of 700 nucleotides—aligned with common practice in the competition, where sequences longer than 700 were often padded due to practical limitations. This constraint may have affected generalization to longer RNAs. Moreover, the need to create our custom datasets may have impacted our meta-learning ensemble due to potential data contamination from the RibonanzaNet2 model. While we engineered features to mitigate this, the model’s calibration was still affected, highlighting the challenges of ensembling with partially unreliable data sources.

Despite these limitations, the consistent trends across evaluations suggest our results remain informative and generalizable.

## 7 Conclusion

This work successfully demonstrated two effective, resource-aware strategies for advancing RNA 3D structure prediction. First, by developing a custom pipeline to integrate Multiple Sequence Alignment (MSA) data and fine-tuning the generalist Protenix model, we achieved a substantial performance gain of nearly 0.08 TM-score, validating the efficacy of targeted, domain-specific adaptation. Second, while simple heuristic ensembles proved insufficient, our feature-engineered meta-learning approach successfully learned to rank and select candidates from a diverse pool of models. This learned ensemble improved the baseline TM-score by over 0.04 points and achieved 94% of the theoretical oracle score, highlighting the power of combining complementary model strengths through a data-driven strategy. Future work should focus on applying this meta-learning framework to our improved fine-tuned models, presenting a clear path toward further state-of-the-art performance.

## Source Code

We have uploaded all the notebooks and code used as part of the Kaggle competition to the following repository: [https://github.com/GravityBoi/3D\\_RNA\\_Project](https://github.com/GravityBoi/3D_RNA_Project)



## References

- Josh Abramson, Jonas Adler, Jack Dunger, Richard Evans, Tim Green, Alexander Pritzel, Olaf Ronneberger, Lindsay Willmore, Andrew J Ballard, Joshua Bambrick, et al. 2024. Accurate structure prediction of biomolecular interactions with alphafold 3. *Nature*, 630(8016):493–500.
- Akash Bahai, Chee Keong Kwoh, Yuguang Mu, and Yinghui Li. 2024. Systematic benchmarking of deep-learning methods for tertiary rna structure prediction. *PLOS Computational Biology*, 20(12):e1012715.
- Clément Bernard, Guillaume Postic, Sahar Ghanay, and Fariza Tahi. 2024. [State-of-the-rnart: benchmarking current methods for rna 3d structure prediction](#). *NAR Genomics and Bioinformatics*, 6(2):lqae048.
- Shujun He, Rui Huang, Jill Townley, Rachael C. Kretsche, Thomas G. Karagianes, David B.T. Cox, Hamish Blair, Dmitry Penzar, Valeriy Vyaltev, Elizaveta Aristova, Arsenii Zinkevich, Artemy Bakulin, Hoyeol Sohn, Daniel Krstevski, Takaaki Fukui, Fumiya Tatsumatsu, Yusuke Uchida, Donghoon Jang, Jun Seong Lee, Roger Shieh, Tom Ma, Eduard Martynov, Maxim V. Shugaev, Habib S.T. Bukhari, Kazuki Fujikawa, Kazuki Onodera, Christof Henkel, Shlomo Ron, Jonathan Romano, John J. Nicol, Grace P. Nye, Yuan Wu, Christian Choe, Walter Reade, Eterna participants, and Rhiju Das. 2024. [Ribonanza: deep learning of rna structure through dual crowdsourcing](#). *bioRxiv*.
- Shujun He, CASP16 organizers, CASP16 RNA experimentalists, RNA-Puzzles consortium, VFOLD team, Rachael Kretsche, Alissa Hummer, Andrew Favor, Walter Reade, Maggie Demkin, Rhiju Das, et al. 2025. Stanford rna 3d folding. <https://kaggle.com/competitions/stanford-rna-3d-folding>. Kaggle.
- Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. 2022. Lora: Low-rank adaptation of large language models. *ICLR*, 1(2):3.
- Kathleen A Leamy, Sarah M Assmann, David H Mathews, and Philip C Bevilacqua. 2016. Bridging the gap between in vitro and in vivo rna folding. *Quarterly reviews of biophysics*, 49:e10.
- Yang Li, Chen-Jie Feng, Xi Zhang, and Yang Zhang. 2025. Ab initio rna structure prediction with composite language model and denoised end-to-end learning. *bioRxiv*, pages 2025–03.
- Valerio Mariani, Marco Biasini, Alessandro Barbato, and Torsten Schwede. 2013. lddt: a local superposition-free score for comparing protein structures and models using distance difference tests. *Bioinformatics*, 29(21):2722–2728.
- Chandran Nithin, Sebastian Kmiecik, Roman Błaszczuk, Julita Nowicka, and Irina Tuszyńska. 2024. Comparative analysis of rna 3d structure prediction methods: towards enhanced modeling of rna–ligand interactions. *Nucleic Acids Research*, 52(13):7465–7486.
- Robert Schmirler, Michael Heinzinger, and Burkhard Rost. 2024. Fine-tuning protein language models boosts predictions across diverse tasks. *Nature Communications*, 15(1):7407.
- Tao Shen, Zhihang Hu, Siqi Sun, Di Liu, Felix Wong, Jiuming Wang, Jiayang Chen, Yixuan Wang, Liang Hong, Jin Xiao, et al. 2024. Accurate rna 3d structure prediction using a language model-based deep learning approach. *Nature Methods*, pages 1–12.
- Marcell Szikszai, Marcin Magnus, Sachin Kadyan, and Elena Rivas. 2025. On inputs to deep learning for rna 3d structure prediction. *bioRxiv*, pages 2025–02.
- Serge Thill. 2025. Deepfoldrna: A graph neural network for rna 3d structure prediction. *Bioinformatics and Code*, 1(1).
- Jiuming Wang, Yimin Fan, Liang Hong, Zhihang Hu, and Yu Li. 2025. Deep learning for rna structure prediction. *Current Opinion in Structural Biology*, 91:102991.
- Wenkai Wang, Chenjie Feng, Renmin Han, Ziyi Wang, Lisha Ye, Zongyang Du, Hong Wei, Fa Zhang, Zhenling Peng, and Jianyi Yang. 2023. [trrosettarna: automated prediction of rna 3d structure with transformer network](#). *Nature communications*, 14(1):7266.
- Chenyue Zhang, Qinxin Wang, Yiyang Li, Anqi Teng, Gang Hu, Qiqige Wuyun, and Wei Zheng. 2024. The historical evolution and significance of multiple sequence alignment in molecular structure and function prediction. *Biomolecules*, 14(12):1531.
- Li Zhou, Xunxun Wang, Shixiong Yu, Ya-Lan Tan, and Zhi-Jie Tan. 2022. Febrna: an automated fragment-ensemble-based model for building rna 3d structures. *Biophysical Journal*, 121(18):3381–3392.

## A Challenges with Competition Evaluation Platform

Our experimental plan was contingent on the ability to evaluate our models on the hidden test set after the final submission deadline of May 29, 2025. This expectation was based on explicit communication from the competition host in the official forums, who stated that submissions would remain open for scoring, albeit without affecting the final leaderboard standings.<sup>1</sup>

However, following the deadline, the submission system was unexpectedly disabled. While subsequent posts from the host on the competition discussion board suggested that this was a temporary measure pending "behind-the-scenes cleanup,"<sup>2</sup> the system remained inaccessible.

To gain clarity, we reached out to the competition host directly. We received definitive confirmation via personal communication on June 16, 2025, that submissions would, in fact, not reopen until the final evaluation period concludes in September 2025, due to an unchangeable platform setting.

This unforeseen logistical barrier made it impossible to score our fine-tuned models on the official test set within the timeframe of our project. This necessitated the creation of our own custom validation set to ensure the scientific validity of our results, as described in the main body of this report.

## B Protenix Fine-Tuning Details

This appendix details the fine-tuning procedure for our custom Protenix model.

### B.1 Training Configuration

The model was fine-tuned using the `torchrun` distributed training utility. The key hyperparameters and loss function weights are summarized in Table 6 for reproducibility.

The loss function was configured to prioritize structural accuracy and physical realism, with the following key weights:

- `loss.weight.alpha_diffusion`: 4.0 (main structure generation loss)
- `loss.weight.alpha_bond`: 1.0 (bond length geometry loss)

<sup>1</sup><https://www.kaggle.com/competitions/stanford-rna-3d-folding/discussion/578554>

<sup>2</sup><https://www.kaggle.com/competitions/stanford-rna-3d-folding/discussion/582324>

Table 6: Key hyperparameters for Protenix fine-tuning.

| Hyperparameter                | Value                        |
|-------------------------------|------------------------------|
| Pre-trained Checkpoint        | <code>model_v0.2.0.pt</code> |
| Learning Rate                 | $1 \times 10^{-4}$           |
| Warmup Steps                  | 1,000                        |
| Effective Batch Size          | 32                           |
| Training Crop Size            | 256 nucleotides              |
| EMA Decay                     | 0.999                        |
| Precision                     | <code>bfloat16</code>        |
| Loss Weight                   | Value                        |
| <code>alpha_diffusion</code>  | 4.0                          |
| <code>alpha_bond</code>       | 1.0                          |
| <code>alpha_distogram</code>  | 0.03                         |
| <code>alpha_confidence</code> | $1 \times 10^{-4}$           |

- `loss.weight.alpha_distogram`: 0.03 (distogram prediction loss)
- `loss.weight.alpha_confidence`:  $1 \times 10^{-4}$  (confidence prediction loss)

### B.2 Training Dynamics and Observations

The training and validation loss curves are shown in Figure 3. The training loss demonstrates a consistent downward trend throughout the fine-tuning process, indicating that the model continued to learn from the new data. A notable observation from the validation loss curve is the behavior of the EMA weights. The validation loss calculated from the raw model weights consistently outperformed the loss from the EMA-smoothed weights.

### B.3 Data Configuration

The model was trained on a custom dataset configuration derived from the official Kaggle RNA 3D folding data. During training, we employed a mixed cropping strategy with weights of 0.2 for contiguous cropping and 0.4 for both spatial and spatial interface cropping to ensure the model was exposed to diverse structural contexts. For the validation set, only the top-1 ground truth structure was used for evaluation. Metal ions were removed from all structures during the cropping process.

The RNA MSA pipeline was configured to use a maximum of 512 sequences for training and 2048 for testing, with a sampling cutoff of 64 sequences fed into the model during training.

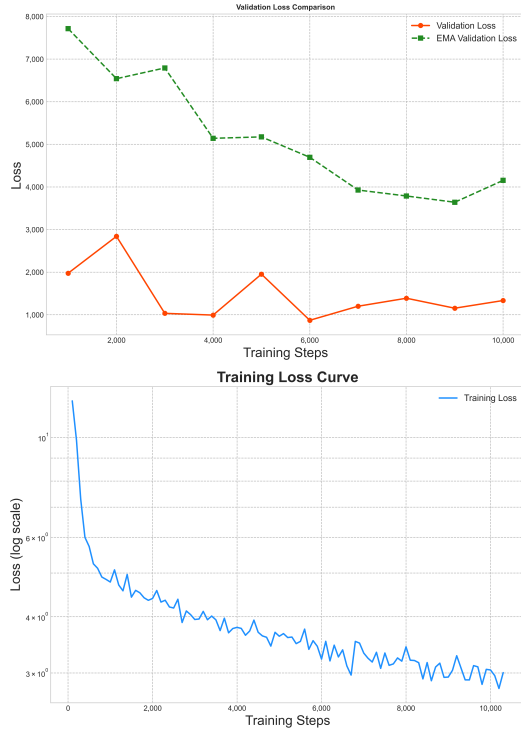


Figure 3: Validation (top) and training (bottom) loss curves during Protenix fine-tuning. The validation plot highlights the divergence between raw model weights and the EMA weights, while the training plot shows steady learning.

## C Final Meta-Model Feature Descriptions

This appendix provides a comprehensive description of the 28 input features used by our final, regularized meta-learning model for TM-score prediction. The features are grouped by category, reflecting the design principles of using model-agnostic, ensemble-based, and targeted interaction signals.

### C.1 Sequence-Based Features

These features capture the fundamental properties of the RNA sequence itself.

- `sequence_length`: The total number of nucleotides in the sequence.
- `percent_A`, `percent_C`, `percent_G`, `percent_U`: The fractional composition of each of the four standard nucleotides.

### C.2 Model-Provided Features

These features are taken directly from the output of the base prediction models.

- `std_plddt`: The standard deviation of the per-residue pLDDT confidence scores. Our analysis showed this was a more informative

signal of structural uncertainty than the mean pLDDT.

- `ptm`: The predicted TM-score, when provided by the base model (e.g., Protenix).
- `ranking_score`: The internal quality or ranking score, when provided by the base model.

### C.3 Ensemble and Structural Features

These features evaluate a candidate structure in the context of the full prediction ensemble.

- `avg_rmsd_to_others`: The average root-mean-square deviation (RMSD) of a candidate to all other 14 candidates for the same target, measuring its agreement with the consensus.
- `is_outlier`: A binary flag (1 or 0) indicating if the candidate was classified as a structural outlier by the DBSCAN clustering algorithm.
- `cluster_size`: The number of structures in the same structural cluster as the candidate. A larger size indicates stronger consensus.
- `num_clusters_total`: The total number of distinct structural clusters found for the target.
- `rog_normalized`: The structure’s radius of gyration normalized by the square root of its sequence length, providing a length-independent measure of compactness.

### C.4 Physics-Based Energy Features

These features are derived from a custom, streamlined energy function designed to capture key indicators of a physically plausible fold.

- `e_vdw`: The Van der Waals energy, primarily acting as a penalty for steric clashes. High positive values indicate a physically unlikely structure.
- `e_solvation`: A simplified solvation energy based on residue exposure. More negative values are favorable and indicate a well-packed, compact core.
- `e_base_pair`: A pseudo-energy term rewarding the proximity of canonical base pairs (AU, GC, GU).
- `e_balance`: The ratio of favorable (solvation, base pairing) to unfavorable (VdW

clashes) energy. This is a powerful metric for overall structural quality.

- `e_attractive_per_res`: The total favorable energy, normalized by sequence length.
- `e_repulsive_per_res`: The total unfavorable (clash) energy, normalized by sequence length.

### C.5 Targeted Interaction Features

These features are explicitly designed to mitigate the data contamination issue from the Ribonanza model by allowing the meta-learner to calibrate its interpretation of a metric based on the source model. For each base model (Protenix, DrFo2, Ribonanza), we create a specific version of key features.

- `std_plddt_x_[model]`: The standard deviation of pLDDT, active only if the structure is from the specified model. (3 features)
- `rog_norm_x_[model]`: The normalized radius of gyration, active only for the specified model. (3 features)
- `low_conf_x_[model]`: The fraction of residues with pLDDT < 70, active only for the specified model. (3 features)

### C.6 Total Feature Count

The final model uses a total of 28 features, as summarized below:

- 5 sequence features
- 3 model-provided features
- 5 ensemble and structural features
- 6 physics-based energy features
- 9 targeted interaction features (3 features for each of the 3 models)

All 28 features are standardized prior to training using a z-score normalization.

## D Heuristic Ensemble formulas

**Naive Ensemble** This strategy selects the first three predictions from Protenix and the first two from DRfold2. The order of the predictions is determined only by the order in which the model makes the predictions, not by confidence scores.

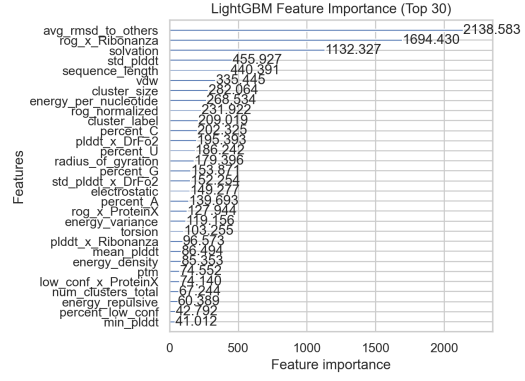


Figure 4: lightgbm model feature list.

**Simple Average Ensemble** This strategy computes the average of all predictions across all models without weighting:

$$\mathbf{x}_i^{\text{ensemble}} = \frac{1}{M} \sum_{m=1}^M \mathbf{x}_i^{(m)}$$

**Weighted Average Ensemble** This approach computes a confidence-weighted average of model predictions:

$$\mathbf{x}_i^{\text{ensemble}} = \frac{\sum_m c_i^{(m)} \cdot \mathbf{x}_i^{(m)}}{\sum_m c_i^{(m)}}$$

**Best Confidence Ensemble** The best confidence ensemble strategy selects the prediction with the highest confidence for each nucleotide:

$$\mathbf{x}_i^{\text{ensemble}} = \mathbf{x}_i^{(m^*)}, \quad m^* = \arg \max_m c_i^{(m)}$$

**Squared Weights Ensemble** This method enhances the standard confidence-weighted average by **squaring the confidence scores**, which increases the influence of highly confident models and reduces the contribution of uncertain ones.

Let  $c_m$  be the confidence score of model  $m$ .

1. Compute amplified weights,  $w_m = c_m^2$
2. Normalize the weights:

$$\alpha_m = \frac{w_m}{\sum_{m' \in M} w_{m'}}$$

3. Compute the weighted average of coordinates

$$\mathbf{r}_{ij}^{\text{ens}} = \sum_{m \in M} \alpha_m \cdot \mathbf{r}^{(m)}$$

This squared weighting amplifies small differences in confidence scores meaning that models with slightly higher confidence gain disproportionately more influence.



**Dynamic Ensemble** This ensemble strategy adapts its behavior based on the distribution of model confidence scores for each atom. It uses a rule-based decision pipeline:

1. **High Confidence Detected:** If any model has a confidence score above a fixed threshold  $\tau = 0.7$ , choose the prediction from the model with the highest confidence.

$$m^* = \arg \max_{m \in M} c_m, \quad \mathbf{r}_{ij}^{\text{ens}} = \mathbf{r}^{(m^*)}$$

2. **Similar Confidence Across Models:** If the standard deviation of all model confidences is low ( $\sigma_c < 0.1$ ), use a confidence-weighted average:

$$\alpha_m = \frac{c_m}{\sum_{m' \in M} c_{m'}}, \quad \mathbf{r}_{ij}^{\text{ens}} = \sum_{m \in M} \alpha_m \cdot \mathbf{r}^{(m)}$$

3. **Default:** If neither of the above conditions is met, default to Protenix):

$$\mathbf{r}_{ij}^{\text{ens}} = \mathbf{r}^{(\text{Protenix})}$$

Protenix was chosen as the default model because it performs the best overall.

## E Custom Dataset Analysis

Table 7: Sequence Length Statistics for Training and Validation Sets

| Statistic    | Training Set | Validation Set |
|--------------|--------------|----------------|
| Count        | 760          | 94             |
| Mean         | 65.38        | 146.62         |
| Std Dev      | 76.24        | 165.99         |
| Min          | 3            | 15             |
| 25%          | 22           | 66             |
| 50% (Median) | 37           | 77             |
| 75%          | 77           | 135            |
| Max          | 720          | 700            |

Our custom datasets consist of 760 sequences in the training set and 94 in the validation set. In Table 7 we detail the sequence length distributions for these custom training and validation sets, which we created by splitting the provided original training dataset. We can see the maximum and minimum sequence lengths are similar, with the validation distribution having a higher mean and median. The bar graphs in figures 5 and 6 show the distribution of all sequence lengths in the training and

validation datasets respectively. Conversely, the boxplots in figures 7 and 8 exclude the outliers. Lastly, figures 9 and 10 show the frequency of each nucleotide in the training and validation dataset respectively.

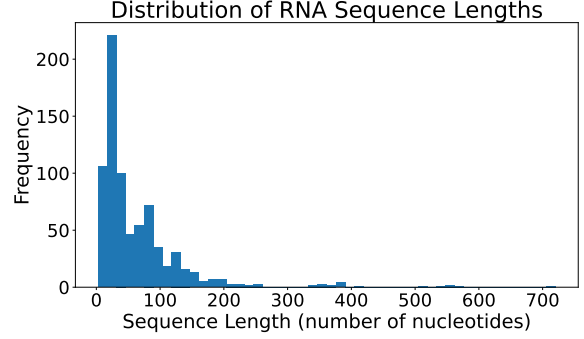


Figure 5: Training data sequence length.

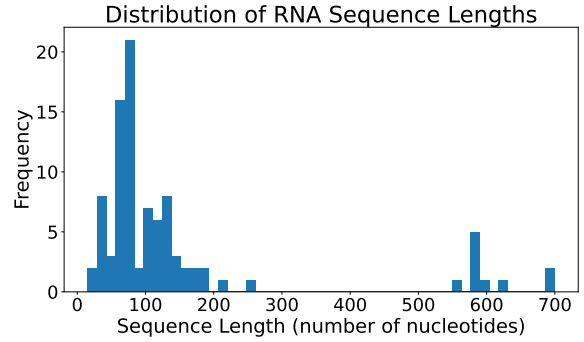


Figure 6: Validation data sequence length.

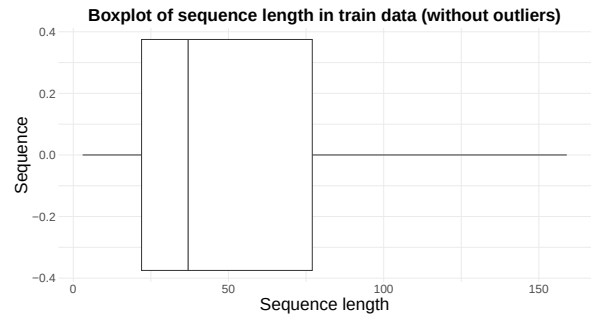


Figure 7: Training data sequence length box plot, excluding outliers.

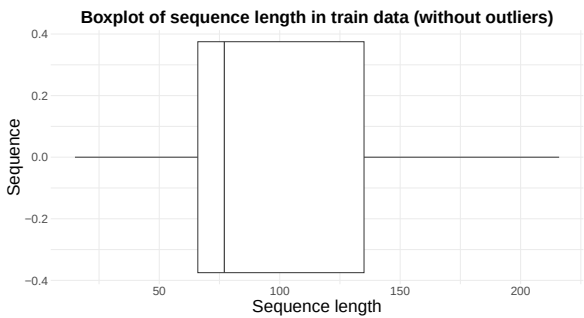


Figure 8: Validation data sequence length box plot, excluding outliers.

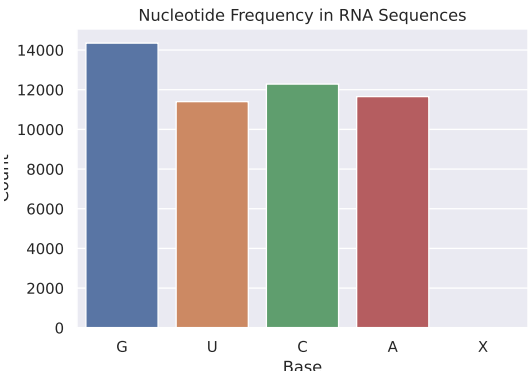


Figure 9: Training data nucleotide frequency

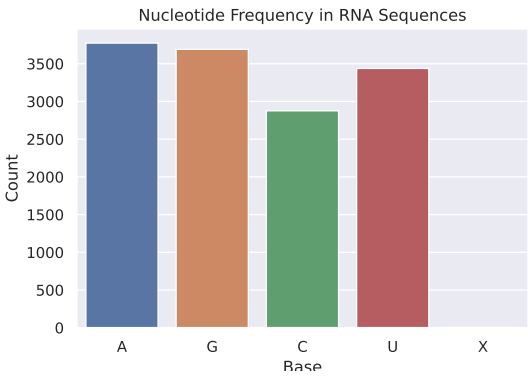


Figure 10: Validation data nucleotide frequency